PorkSort: Pass/Fail Inspection of Pork Rasher Packaging

Hashim Navas · 33856354 · MMA3001 Numerical Methods and Machine Learning, Semester 2 2026 ·
Repository: github.com/hashim-edu/mma3001-project-pork_sort

1 Engineering problem

Pork rashers are packed in plastic trays and sealed with lidding film on automated tray-sealing lines, which run at about 120 packs per minute [1] and up to 200 on recent machines [2]. An unsealed or damaged pack risks spoilage and a recall, and loose or twisted meat leads to complaints. At two to three packs per second, people cannot inspect every pack consistently over a shift, so automating the check is a machine-vision problem within the instrumentation and control of the line.

The Dataset 3 brief poses an open problem, a report on meat and packaging quality, and asks for a narrower, achievable one. This system takes an RGB frame from the overhead camera, decides whether the visible tray should PASS or FAIL packaging inspection, and shows where any defect is.

The intended use is decision support: every FAIL frame is shown to an operator with its defect boxes, and the operator makes the final call. The system never rejects the product on its own. The scope is limited by what the data can support:

- *Per frame, not per tray:* Frames carry no tray identifier, and a tray stays in view for about 12 frames.
- *Packaging and placement only:* The labels cover five defects (unsealed film, other packaging errors, loose meat, twisted meat and wrinkles) and nothing about meat quality.

FAIL means any of the five defects are present; all five are treated as equally serious, because the safe response to an unconfirmed defect is a human check.

2 Inputs, outputs and data

2.1 Top-level inputs and outputs

The solution accepts image files, or folders of them, each an overhead frame of one tray (Table 1).

Table 1. Top-level inputs and outputs.

Item	Format and units	Domain	Meaning and use
Input frame	JPEG, PNG or BMP; RGB, 8 bits per channel	720 x 540 px overhead conveyor frame. Other sizes are letterboxed and flagged	One camera frame of the line
Defect score s	Dimensionless	0 to 1	Highest confidence among detected defect boxes; ranks how suspicious the frame is
Threshold τ	Dimensionless	0 to 1, fixed from validation data for a cost ratio r	The frame FAILs when $s \geq \tau$
Status	Text	PASS, FAIL or ERROR	Operator action: let through, re-inspect, or check the camera or file
Defects	Class; confidence 0 to 1; box corners in px; area as a fraction of the frame	Five classes	What is wrong and where to look
Timing	Milliseconds	Positive	Load, preprocess, inference, postprocess and total time; checks the line budget
Warnings	Text	Wrong size, very dark (mean below 15/255), very bright (above 240/255), near-uniform	The input is unlike the training data

Each frame produces a JSON report and an annotated PNG, and a batch adds a summary and an HTML index. Invalid inputs never stop a batch: a file that is not an image, or cannot be decoded, gets status ERROR with the reason, and grey or RGBA images are converted to RGB. The command-line tool exits with code 2

if any frame was an ERROR and 1 for invalid arguments, which a test checks with good, corrupt and non-image files. Appendix B gives the inputs, outputs and errors of the main functions, and the HTML documentation collects every function's docstring.

2.2 The data

The data is the Roboflow "Pork Rasher Error(Packaging)" export, version 4, under CC BY 4.0 [3]: 3,549 images from 1,272 distinct frames of one overhead conveyor video, with boxes for the five defect classes. Frames were already resized to 720 x 540 px by centre crop, and each training frame has three copies with exposure shifted by up to ±10%. Labelling a frame FAIL if it has any defect box makes PASS the minority, about 20% of images (Figure C3). Twisted meat, loose meat and wrinkle each appear in fewer than 70 training images, and wrinkle in no test frame. The median packaging-error box covers 0.38% of the frame (about 37 x 37 px), the median unsealed box 10.9%.

2.3 Training, validation and test: the blocked split

The training set fits each model, the validation set is used for every choice, and the test set is used once to report performance, which only works if the three are independent. The provided split is not: all 80 of its test frames have a training frame within 30 frame numbers, and 23 within one (Figure C1). Neighbouring frames show the same tray slightly moved, so testing on them partly tests on trays seen in training, a form of leakage [4]. Thumbnail correlation repeats every 11.9 frames, found from the FFT of the similarity-against-gap curve (Figure C2): that is the tray spacing, so frames 30 or more apart show different trays.

The blocked split assigns whole blocks of 200 consecutive frame numbers to train, validation or test, as recommended for temporally structured data [5], and drops any frame within 30 frame numbers of a higher-priority split. Of 500 random block assignments I kept the one whose validation and test label mix was closest to the whole dataset, using labels only, never model results. The result is 869 training frames (2,433 images with copies), 192 validation and 174 test frames, of which 37 and 34 are PASS, with one copy of each frame in validation and test.

3 Computational approach

Every model is reduced to one defect score, s, between zero and one per frame, and the frame FAILs when s ≥ τ, so all models are judged by the same rule and threshold procedure (Figure 1).

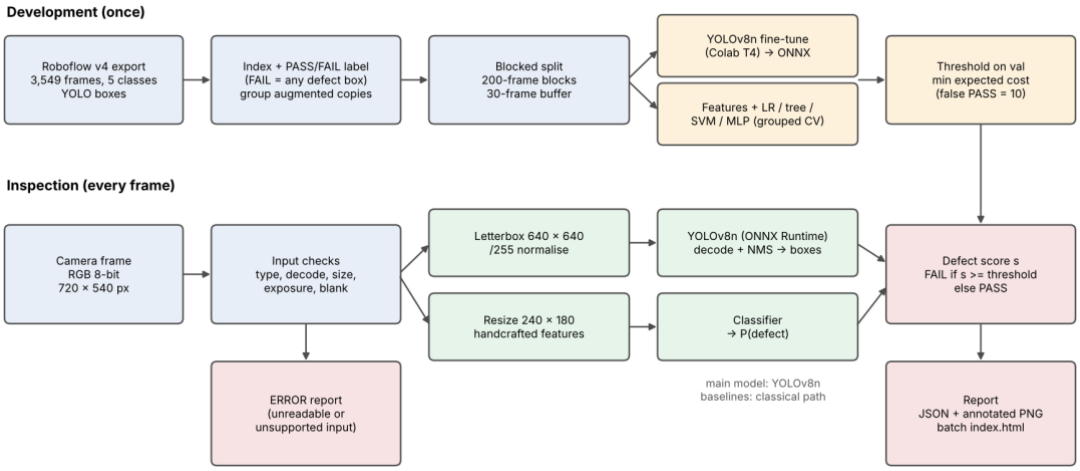


Figure 1. PorkSort architecture. Development (top) runs once; inspection (bottom) runs on every frame. The classical path is the baseline family the selected YOLOv8n detector was compared against.

3.1 Classical models on handcrafted features

A frame has over a million pixel values and there are only 869 distinct training frames, so the classical models work on 1,439 features of the frame resized to 240 x 180 px: a joint HSV colour histogram; the share, centroid and spread of meat-coloured pixels; glare on a 4 x 3 grid; Sobel edge strength on a 6 x 4 grid; a local binary pattern histogram [6]; and a histogram of oriented gradients (HOG) [7]. The edge and HOG features rest on finite-difference image gradients. Four classifiers, the classification counterparts of regression models from the unit [8], were trained on them: logistic regression, a decision tree, an RBF support vector machine (SVM) and a multilayer perceptron (MLP), using scikit-learn [9] and scikit-image [10]. Features were standardised, since otherwise large-range features dominate the SVM's distances and make gradient descent badly conditioned, and classes were weighted by inverse frequency. Hyperparameters were chosen by five-fold cross-validation scored by the Receiver Operating Characteristic-Area Under the Curve metric (ROC AUC), with folds grouped by 200-frame block (Appendix D).

3.2 YOLOv8n detector

The smallest defects are about 37 px wide and global features cannot see them (Section 5.3), so a detector that examines every part of the image is the natural tool. YOLOv8n was chosen as it is the smallest YOLOv8 model [11], [12]: it is fast enough for a CPU, its pretrained weights transfer to a small dataset, and its boxes explain each FAIL. A convolutional backbone produces feature maps at three scales, and a head predicts a box and five class scores at each of 8,400 positions for a 640 px input. Boxes below a confidence threshold are dropped, and non-maximum suppression (NMS) keeps the highest-scoring box and deletes same-class boxes whose intersection over union (IoU) with it exceeds 0.6.

COCO-pretrained weights were fine-tuned on a free Colab T4 GPU at a 640 px input (frames letterboxed to 640 x 480 plus padding), batch 16, seed 0, default augmentation and up to 60 epochs, an epoch being one pass through the training images. Early stopping ended training when validation mean Average Precision (mAP) had not improved for 20 epochs: it stopped at epoch 39 after 22 minutes, keeping the epoch-19 weights. The network was exported to ONNX and runs with ONNX Runtime on a CPU [13]; letterboxing, decoding and NMS are the package's own NumPy code, so inspection needs neither PyTorch nor a GPU (`python -m porksort inspect`). The deployed model runs at 480 px (Section 6.3).

3.3 Choosing the threshold by cost

A threshold of 0.5 has no special meaning here, because the two errors cost different amounts: a defective pack that passes risks a recall, a good pack that fails costs a re-check. The threshold τ was chosen to minimise the expected cost per frame on the validation set:

$$C(\tau) = \frac{r \cdot \mathrm{FN}(\tau) + \mathrm{FP}(\tau)}{N}$$

FN counts defective frames passed, FP good frames failed, and N all frames. I set r = 10, so a shipped defect counts as ten re-checks (Section 6.4 tests this). The candidates are the midpoints between consecutive distinct scores, so the search finds the exact minimum on the data; ties go to the lower threshold. The deployed thresholds are 0.007 at r = 10 and 0.128 at r = 1. The method assumes that the annotations are correct and complete, that new frames come from the same line, camera and lighting, that the five classes are equally serious, and that frames can be judged independently. The limitations of this methodology are further explored in Section 6.6.

4 Alternative solutions

The final design was compared with a trivial baseline and four classical alternatives. Every model turns a frame into a defect score and uses the same cost-based threshold (Section 3.3), so differences in the results come from the model alone. The baseline, "always FAIL", is what the line does without the system: re-check every pack. The classical models (logistic regression, a decision tree, an SVM and an MLP) are simpler

methods taught during MMA3001 that judge the whole frame from handcrafted features (Section 3.1). The detector, YOLOv8n, searches the frame for each defect and draws a box around it. Table 2 compares them on the brief's criteria; AUC measures how well a model ranks defective frames above good ones, where 1 is perfect and 0.5 is guessing (Section 5.2).

Table 2. Alternatives compared on the brief's criteria. Times are per frame on the M4 laptop CPU unless marked.

Model	Blocked test AUC	Time per frame	Model size	Interpretability	Localises defects
Logistic regression	0.739	2.8 ms features (optimised) + 0.25 ms (container)	about 35 kB	High: one weight per feature	No
Decision tree (depth 5)	0.717	features + 0.11 ms (container)	under 10 kB	High: readable rules	No
SVM (RBF)	0.724	features + 2.0 ms (container)	1,606 support vectors	Low	No
MLP (64 hidden units)	0.717	features + 0.31 ms (container)	about 0.75 MB	Low	No
YOLOv8n, 480 px	**0.887**	36.7 ms inference, 41 ms end to end	12.7 MB (fp32), 6.4 MB (fp16)	Medium: boxes show why	Yes

YOLOv8n at a 480 px input was selected as it ranks frames best: its margin over logistic regression, 0.148 AUC, has a paired 95% interval of 0.068 to 0.235, which excludes zero. It is the only model that shows the operator where to look, its score was not inflated by the leaky provided split (Section 5.4), and at 41 ms per frame it is fast enough for the line (Section 6.4). The price is a file about 360 times larger than logistic regression's, GPU training, a more complex pipeline to implement, and internals that are harder to inspect. The simpler models would be preferable if they ranked as well, but whole-frame features miss small defects (Section 5.3). YOLOv8n is the best option tested, not one that meets the acceptance criteria.

5 Validation

5.1 What was validated and verified

Validation asks whether the system makes the right decisions on frames it has never seen; verification asks whether the code does what is intended. For validation, the decision for each of the 174 blocked test frames (Section 2.3) was compared with its human label once, after every model, setting and threshold had been fixed, so the test result could not influence any choice.

For verification, the repository has 52 automated tests (Appendix E). Four compare the code with answers worked out in advance, like a manufactured solution: the decoder recovers a hand-encoded box to within 1e-5, the FLOP formula matches a direct loop count, the threshold search matches a 20,001-point brute-force search, and the optimised histogram is bit-identical to NumPy's. I also compared my ONNX pipeline with an established implementation, Ultralytics' own predictions, on 60 test frames: 34 of the 41 boxes it found at confidence 0.25 or more matched mine at IoU $\geq$ 0.9 (median 0.94), and 46 of 53 for the second model of Section 5.4. The remaining score differences (0.04 on average) come from how the frame is prepared for the network, not from my decoding (Appendix G).

5.2 Why this method, the metrics and the acceptance criteria

Withheld test data is the most direct check on new frames, and the blocked split keeps every test tray out of training. Cross-validation, the Ultralytics comparison, known-answer tests, repeated runs, sensitivity studies (Section 6) and inspection of individual decisions (Figure 3) support it. Data from another line was not available.

The system can make two kinds of mistake, measured separately with FAIL as the positive class. An escape is a defective pack that is passed; the escape rate, FN / (TP + FN), is the primary metric because escapes are the costly mistake. A false reject is a good pack sent for an unnecessary re-check; the false reject rate is FP / (FP + TN). ROC AUC, computed by the trapezoidal rule, is the probability that a random defective frame

scores higher than a random good one, so it measures ranking across all thresholds. Accuracy is not used: 80.5% of test frames are defective, so always answering FAIL would score 80.5% while being useless.

With only 34 good test frames the rates are uncertain, so each has a 95% Wilson interval [14], which unlike the normal approximation stays valid at 0% and 100%; AUC intervals come from 2,000 bootstrap resamples of the test set. The two acceptance criteria established for this system, set before any detector result was seen, were: an escape rate with an upper 95% bound of at most 5%, and a false reject rate of at most 30%. Validation data shaped the design: cross-validation chose the classical hyperparameters, and the validation set chose the threshold, input size, stopping epoch and final model. The test set was used once.

5.3 Results

Table 3. Blocked test set (174 frames, 34 PASS), thresholds chosen on validation at r = 10. Intervals are 95%.

Model	ROC AUC (95% CI)	Threshold	Escape rate	False reject rate	Cost per frame
Always FAIL (baseline)	0.500	0	0.000	1.000	0.195
Colour-histogram logistic regression	0.739 (0.652 to 0.822)	0.205	0.000	0.941	0.184
Logistic regression	0.739 (0.649 to 0.823)	0	0.000	1.000	0.195
Decision tree	0.717 (0.621 to 0.807)	0.147	0.029	0.971	0.420
SVM (RBF)	0.724 (0.633 to 0.806)	0	0.000	1.000	0.195
MLP	0.717 (0.618 to 0.805)	0	0.000	1.000	0.195
YOLOv8n, 640 px input	0.859 (0.773 to 0.926)	0.004	0.000	1.000	0.195
YOLOv8n, 480 px input (selected)	**0.887 (0.810 to 0.947)**	0.007	0.000	1.000	0.195

The detector ranks frames far better than any classical model (Table 3, Figure 2), 0.148 AUC above logistic regression on the same frames (paired 95% interval 0.068 to 0.235). The gain is in the small defects: scored against the PASS frames, its AUC is 0.95 for unsealed film and 0.84 for packaging errors, against 0.86 and 0.67 for logistic regression, and 0.71 and 0.97 for the rare loose and twisted meat classes (six and four frames), against 0.51 and 0.49. Whole-frame features see large defects but not small ones.

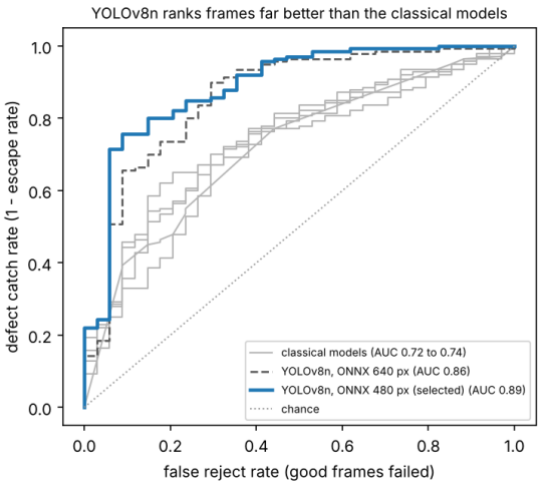


Figure 2. ROC curves on the blocked test set. The selected detector (blue) dominates every classical model (grey).

Ranking well is not the same as deciding well. At r = 10, passing a good pack saves one re-check but passing a defective one costs ten. With 80% of frames defective, and defective frames among even the lowest-scoring validation frames (including five wrinkle frames, a class with three training frames), the detector's

cheapest threshold fails every frame. The best any model does is the colour-histogram model, passing 2 of 34 good frames, and none meets the acceptance criteria. Table 4 shows where the detector does pay off.

Table 4. Selected detector: operating points chosen on validation, evaluated on test.

Cost ratio r	Threshold	Escape rate (95% CI)	False reject rate (95% CI)	Cost per frame (re-check all: 0.195)
1	0.128	3.6% (1.5% to 8.1%)	44% (29% to 61%)	0.115
2 to 50	0.007	0%	100%	0.195

At r = 1, where a miss costs the same as a re-check, the detector passes 19 of 34 good frames and lets 5 of 140 defective frames through, cutting the cost per frame by 41% (Figure 3). Most escapes are small packaging errors at tray corners and loose pieces at the frame edge; several false rejects are trays whose film looks lifted but carries no label, possibly a labelling ambiguity.

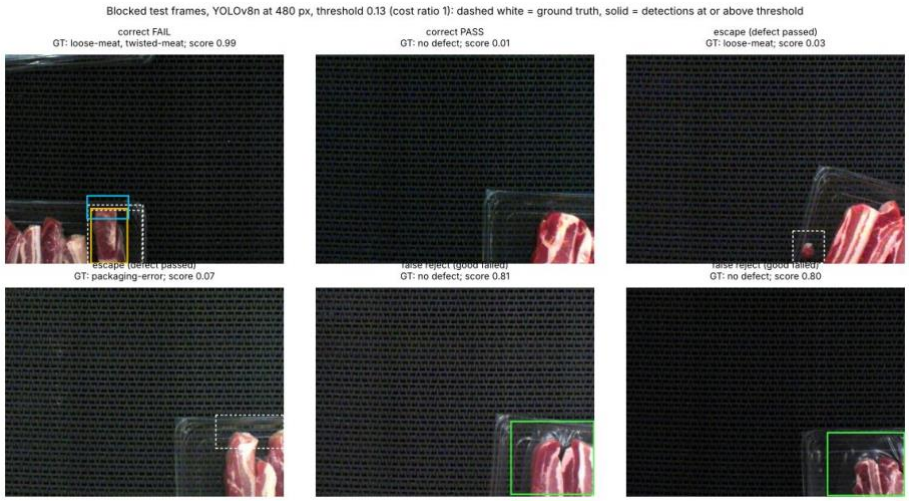


Figure 3. Blocked test frames at the r = 1 threshold. Dashed white boxes are labels; solid boxes are detections.

5.4 Effect of the split: leakage comparison

The whole process was repeated on the provided split to measure how much it would have flattered the results, including a second 46-minute YOLOv8n training run. Test AUC rose by 0.069 for logistic regression (0.808 against 0.739) and 0.102 for the SVM (0.826 against 0.724), but not measurably for the detector: 0.885 (95% CI 0.761 to 0.973) against 0.887 (Figure C7). A likely reason is that whole-frame features let the classical models recognise near-duplicate trays, while the detector judges small regions. The leak still affected the detector's training: its validation mAP50 was 0.577 against 0.362 and early stopping ran 20 epochs longer, so a leaky validation set would have misled model selection. The provided test set has only 80 frames (12 PASS), so differences below about 0.1 AUC cannot be detected.

5.5 What the validation cannot establish

Because every frame comes from one recording, the validation cannot show performance on another line, camera, lighting setup or product. It cannot show behaviour per tray, since a defect visible in one frame may be out of view in the next. It says nothing reliable about the rare classes (wrinkle has no test frames; loose and twisted meat have four to six), the effect of labelling errors, or whether r = 10 reflects the plant's real costs.

6 Optimisation and performance

This section asks whether the system is fast and light enough for a packing line, and where its speed or accuracy can be improved without changing its results. Each timing is the median of repeated runs (30 for the detector) on the target laptop, an M4 MacBook running a four-core Linux virtual machine (Appendix F), and every change was compared with the original for speed and numerical effect. The section ends with the limitations (6.6) and lessons learned (6.7).

6.1 Profiling and optimising the baseline pipeline

The classical models spend almost all their time computing features, not classifying. Extraction took 13.9 ms per frame, and line_profiler put RGB-to-HSV colour conversion at 36% of it, the colour histogram at 14%, texture at 13%, HOG at 11% and resizing at 10%. I made four changes, each tested for its numerical effect. Pillow's 8-bit HSV conversion, written in C, replaced scikit-image's 64-bit one; the values differ by at most 1/255, far less than a histogram bin. The histogram now computes each pixel's bin directly and counts with `bincount`, bit-identical to `histogramdd`. An ablation study, removing one feature group at a time, showed only colour and HOG mattered (validation AUC fell by 0.026 and 0.036 without them, under 0.003 for any other group), so the other four groups were dropped. Finally, resizing is 2D interpolation: across three working sizes and four methods (Figure C4), box filtering, which averages the pixels each output pixel covers, was best at every size, while nearest-neighbour turned the belt's fine weave into noise (aliasing). I chose box filtering at 120 x 90 px. Extraction fell from 13.9 to 9.0, 4.9 and finally 2.8 ms, 4.9 times faster, and accuracy rose rather than fell (validation AUC 0.718 to 0.758, test 0.739 to 0.748).

6.2 How much work the detector does: FLOPs and arithmetic intensity

Almost all the detector's work is in its convolutions. Each output value needs one multiply and one add per input value it combines, so a layer costs 2 x C_out x H_out x W_out x (C_in / groups) x k^2 floating-point operations (FLOPs). Summed over the network this gives 8.64 GFLOPs per 640 px frame, within 5% of Ultralytics' 8.2, which checks the count. If each layer reads its weights and input and writes its output once, the network moves 153 MB per frame, so its arithmetic intensity, the FLOPs done per byte moved, is 57.

The roofline model [15] uses this to show what limits speed. A processor's peak arithmetic rate divided by its memory bandwidth is its machine balance: code with a lower intensity waits for memory (memory-bound), and code with a higher one waits for arithmetic (compute-bound). The balance is about 4.7 FLOP/byte for the M4 [16] and 25 for the Colab T4 [17] (Appendix G), so at 57 the network is compute-bound on both (Figure 4): doing less or cheaper arithmetic helps more than faster memory. On the M4 it reached 140 GFLOP/s, a quarter of the peak (Appendix G).

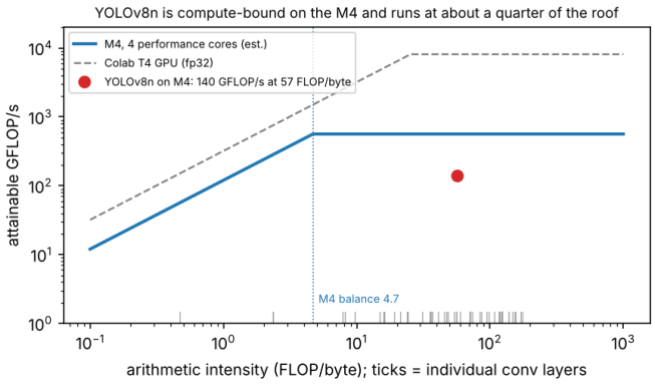


Figure 4. Roofline for the M4 and the T4. Ticks mark individual convolution layers; the red point is the measured whole-network performance on the M4.

6.3 Settings that trade speed against accuracy

Input size is the biggest lever. The detector was trained at 640 px, but a frame can be resized to another size before inference, and as the FLOPs scale with the pixel count, the time roughly does too (Table 5, Figure C5). I chose 480 px on validation: the same AUC as 640 px within noise, in 59% of the time. Its higher test AUC (0.887 against 0.859) is not claimed as a gain, and 800 px did not help the smallest defects, which points to training at a higher resolution instead.

Table 5. YOLOv8n trained at 640 px, run at other input sizes: validation AUC and M4 inference time per frame.

Input size	320 px	480 px	640 px	800 px
Validation AUC	0.848	0.875	0.870	0.851
Inference time per frame	17.7 ms	36.7 ms	61.9 ms	98.5 ms

Precision is the second lever. Storing the network as 16-bit floats (fp16) instead of 32-bit halved the file from 12.8 to 6.4 MB and cut 640 px inference from 61.9 to 42.9 ms. The risk is rounding error, as fp16 keeps about three significant figures (unit roundoff $2^{-11} \approx 0.0005$); the largest frame-score change over 60 frames was 0.0008 and no decision changed. fp16 was exported only at 640 px, so the deployed 480 px model is fp32. Four threads ran 3.3 times faster than one (207 to 62 ms), against an ideal of four.

6.4 Line budget, scalability, robustness and repeatability

At 120 to 200 packs per minute, judging one frame per pack leaves 300 to 500 ms, so 41 ms end to end is 7 to 12 times inside the budget. Judging every frame (11.9 per tray) leaves 25 to 42 ms: 480 px fits only at 120 packs per minute, while 320 px (21 ms) fits both for 0.027 less validation AUC. Frames are independent, so a faster line can use more cores, a GPU or fp16. Memory is not a constraint: the largest model file is 12.7 MB, though peak memory use was not measured.

The decisions are robust to most settings tested. Sweeping r from 1 to 50 (Figure C6) moves the detector's threshold only between r = 1 and 2, and the classical thresholds not at all, so their weakness is ranking, not the choice of r; fp16 and the faster HSV conversion changed no decisions. The detector is sensitive to preprocessing, though: moving the padding alone shifted scores by 0.024 (Appendix G). Timings repeat closely (80% of 480 px runs fell within 35.5 to 37.9 ms), training less so: five MLP seeds gave a test AUC of 0.716 ± 0.008, and each YOLOv8n configuration was trained once. All random choices are seeded, so the classical results regenerate exactly; GPU training may not repeat bit for bit.

6.5 Trade-offs and the final design

Table 6. Trade-offs behind the final design.

Choice	Gain	Cost	Decision
YOLOv8n instead of logistic regression	AUC +0.148; shows where the defect is	About 360 times larger file; GPU training; 36.7 against about 3 ms of computation per frame	YOLOv8n
480 px instead of 640 px	41% less inference time	None measurable on validation	480 px
320 px instead of 480 px	Half the time; fits every frame at 200 packs per minute	0.027 lower validation AUC	Option
fp16 instead of fp32	Half the file; 31% less time at 640 px	Scores change by up to 0.0008; no decision changed	Next step at 480 px
Lean classical features	4.9 times faster extraction	None: AUC rose	Adopted

Overall (Table 6), the final design trades a larger, GPU-trained model for the only ranking that clearly beats the classical models and for defect locations, while staying well inside the line's time budget; 320 px and fp16 remain options for a faster line.

6.6 Limitations

- *It does not meet the acceptance criteria:* At r = 10 the detector is a prioritisation aid with boxes, not an automatic gate.
- *Per-frame decisions:* A frame can be labelled PASS while its defect is just out of view; a deployed system should track each tray and fail it if any of its frames fails.
- *Rare classes:* Wrinkle has three training frames and no test frames, so nothing is claimed about its recall.

- *One recording:* All frames come from one line with an unknown frame rate, and 32 frame names carry two different images, suggesting two sessions.

6.7 Lessons learned

- *The split decides what is measured, though not equally for every model:* it overstated logistic regression and the SVM by up to 0.10 AUC, not the detector, yet still misled the detector's early stopping.
- *Ranking is not deciding:* an AUC of 0.89 still fails a 10 to 1 cost test when 80% of frames are defective.
- *Match the method to the scale of the defect:* global features found unsealed trays but not small local defects.
- *Profile before optimising:* the slowest step was colour conversion, not the model.

Next steps: train at about 960 px for the small packaging errors behind most escapes, collect more rare-class examples, track trays across frames, train several seeds, and replace r = 10 with costs measured at the plant.

7 AI use and reflection

Tools and tasks: I used Claude (Anthropic) through the Claude app, linked to my laptop and a cloud workspace. It wrote most of the code (the package, analysis scripts, tests and Colab notebook), ran the data audit and analyses, and drafted this report from the work and results that I then reviewed against the criteria and expected system behaviour based on my own design. Google Colab provided the training GPU.

Level of contribution and why: High: the AI wrote most of the code and drafted the report scaffold and diagrams, under my direction. I used it because the project needed tools I had not used before (YOLO, ONNX, profiling image code) and this project was undertaken during a busy period of the semester as I had multiple other assessments and was away for about two weeks representing the university at a student engineering competition.

How I directed: I chose the dataset and the per-frame PASS/FAIL goal with defect locations, required methods from the unit outline and chose to train on Colab due to my MacBook lacking a dedicated GPU, ran both training runs and reported their errors back, and decided to train the second detector to complete the leakage comparison. The AI proposed the blocked split, the cost-based threshold, r = 10 and the acceptance criteria; I reviewed and accepted each one and am responsible for them. I checked the scaffold and the outputs against the brief to ensure it was up to standard, and either made relevant fixes myself, and primarily asked Claude to make relevant edits to code systems.

How the output was checked: By 52 automated tests, four against known answers; by comparison with Ultralytics' predictions on both trained models; by looking at sample frames and every figure; and by tracing each number to a file in `results/`. I also personally ran tests myself on my laptop to ensure the system was producing outputs as expected and checked them manually to observe and check its behaviour, deeming it to be correct and consistent with the expected goals of the system and the marking criteria.

Errors and unhelpful output: Its first frame-similarity analysis used a ±2 standard deviation band too wide to show anything (an FFT of the curve gave the tray spacing instead). Its Colab notebook first failed because the code folder had the package's name and hid it, and it once delivered an out-of-date copy of the code, caught by file sizes. Two of its tests had bugs; draft text assumed a 30 fps camera and claimed no model beat re-checking every pack at r = 10, when one passes 2 good frames; and its first full draft was over the page limit and missed function-level inputs and outputs. These erroneous outputs required manual investigation and fixes or redirection to ensure they were functioning as expected.

Effect on my learning and workflow: Using Claude meant I could produce a project that was more complex than my personal knowledge would have allowed, on a timeline that was quite condensed. The clear trade-off being that I was able to implement ideas and features quickly without having to learn the methods of

integration explicitly beforehand, but learning as I went. This was accomplished by asking Claude to explain each system as it was built and specifically having it tailor responses as my knowledge in some areas were lower than others. In doing so, most of my time was spent directing Claude on what features were needed and what kind of outputs these would result in, and learning how each feature was integrated as the project was built.

References

1. "Proseal launch for GT2X Twin E-seal," *BakeryandSnacks.com*, Jul. 2012. [Online]. Available: bakeryandsnacks.com
2. "Harpak ULMA launches high speed grind line tray sealing application powered by Mondini Sinfonia," *Business Wire*, Jan. 2026. [Online]. Available: businesswire.com
3. "Pork Rasher Error(Packaging)," version 4, Roboflow Universe, workspace hello-2aqe0, licensed CC BY 4.0, 2025. [Online]. Available: universe.roboflow.com
4. S. Kaufman, S. Rosset, C. Perlich and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, 2012.
5. D. R. Roberts *et al.*, "Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure," *Ecography*, vol. 40, no. 8, pp. 913-929, 2017.
6. T. Ojala, M. Pietikäinen and T. Mäenpää, "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 7, pp. 971-987, 2002.
7. N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE CVPR*, 2005, pp. 886-893.
8. MMA3001 teaching team, *MMA3001 Numerical Methods and Machine Learning unit notes*, Weeks 2.1, 2.2, 3.5 and 5.1 to 5.6, Monash University, 2026.
9. F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825-2830, 2011.
10. S. van der Walt *et al.*, "scikit-image: Image processing in Python," *PeerJ*, vol. 2, e453, 2014.
11. J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE CVPR*, 2016, pp. 779-788.
12. G. Jocher, A. Chaurasia and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: github.com/ultralytics/ultralytics
13. ONNX Runtime developers, "ONNX Runtime," Microsoft. [Online]. Available: onnxruntime.ai
14. E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209-212, 1927.
15. S. Williams, A. Waterman and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65-76, 2009.
16. "Apple Silicon: M4 chip specs," *Low End Mac*, 2025. [Online]. Available: lowendmac.com
17. NVIDIA, "NVIDIA T4 Tensor Core GPU." [Online]. Available: nvidia.com

Appendix A: Repository guide

Repository: github.com/hashim-edu/mma3001-project-pork_sort. The README explains the purpose, inputs and outputs, installation, and how to reproduce every result: install with `pip install -r requirements.txt` then `pip install -e .`, unzip the Roboflow export to `data/pork_v4/`, and run `01_data_audit.py`, `02_classical_models.py`, the Colab notebook `03_train_yolo_colab.ipynb`, `04_evaluate_and_compare.py`, `05_performance.py` and `06_figures.py` in order. The inspection tool runs with `python -m porksort inspect <frames> --out reports/`; `--cost-ratio 1` selects the threshold chosen on validation for r = 1. Every public function has a Google-style docstring giving its arguments, return value and errors (collected in the HTML documentation), and the code is formatted and linted with ruff.

Folder	Contents
src/porksort/	Package: dataset, splits, features, models, decision, evaluate, detector, perf, report, cli
scripts/	Numbered analysis scripts that write to results/
notebooks/	Colab training notebook
splits/	The committed blocked split, so every machine uses the same frames
results/	Metrics (JSON, CSV), profiles and figures used in this report
tests/	pytest suite on synthetic data; needs no dataset
docs/	HTML API documentation (pdoc, Google-style docstrings) and the pytest HTML test report

Appendix B: Inputs and outputs of the main functions

Boxes are Box records: class name, centre x and y, width and height as fractions of the image in [0, 1], and confidence in [0, 1]. Full docstrings are in docs/api/.

Function	Inputs	Output	Invalid input
dataset.parse_label_file(path)	YOLO label file; one line per box: class id, cx, cy, w, h in [0, 1]	List of Box	FileNotFoundError; ValueError naming the file and line for a wrong field count, unknown class or coordinate outside [0, 1]
dataset.build_index(root)	Export folder with train, valid and test images and labels	DataFrame, one row per image: path, provided split, source frame, frame number, boxes, FAIL label	FileNotFoundError if the folders or a label file are missing
splits.blocked_split(index, cfg)	Image index; BlockedSplitConfig (block 200 frames, gap 30, validation and test fractions 0.15, 500 trials, seed 0)	Each source frame assigned to train, val, test or buffer	ValueError if the fractions leave no training set
features.load_rgb(path)	Image file: .jpg, .jpeg, .png or .bmp	uint8 array (H, W, 3); grey and RGBA converted to RGB	FileNotFoundError; ValueError if unsupported or undecodable
features.extract(rgb, cfg)	uint8 RGB array of any size; FeatureConfig (working size, interpolation, feature groups, HSV method)	float vector of fixed length for a config: 1,439 for all groups, 1,388 for colour and HOG	ValueError if not (H, W, 3)
models.make_model(name)	majority, colour_lr, logreg, tree, svm or mlp	Untrained scikit-learn pipeline and its hyperparameter grid	ValueError for an unknown name
models.defect_score(model, X)	Fitted pipeline; feature matrix (n, d)	n defect scores in [0, 1]	
detector.letterbox(rgb, size, fill)	uint8 RGB frame; square side in px; padding grey level (114)	Padded (size, size, 3) array, scale r, padding (dx, dy) in px	
detector.decode(output, r, pad, img_wh, conf_thr, iou_thr)	Raw network output (1, 9, anchors): cx, cy, w, h in letterboxed px and five class scores in [0, 1]; r and pad from letterbox; frame size; thresholds	Up to 300 Box in normalised frame coordinates, after per-class NMS	
detector.nms(boxes, scores, iou_thr)	Boxes (N, 4) as corner coordinates; scores (N,); IoU threshold	Indices of kept boxes, highest score first	
detector.OnnxYoloDetector(path, conf, iou, threads).detect(rgb)	ONNX model file; uint8 RGB frame	List of Box; per-stage times in ms in last_timing	FileNotFoundError if the model is missing
decision.decide(boxes, threshold)	Detected boxes; τ in [0, 1]	Decision: FAIL if the frame score, the highest defect confidence (0 if none), is at least τ; the score; the defects	ValueError if τ is outside [0, 1]
decision.choose_threshold(y_fail, scores, cost_ratio)	Labels (True = defective), scores in [0, 1], cost ratio r	(τ, minimum expected cost per frame), ties to the lower τ	

Function	Inputs	Output	Invalid input
`evaluate.wilson_ci(k, n)`	Count k of n trials	95% interval (lower, upper) in [0, 1]	
`evaluate.passfail_metrics(y_fail, scores, threshold, cost_ratio)`	Labels, scores, τ, r	Dict: confusion counts, escape and false reject rates with intervals, expected cost, ROC AUC	
`evaluate.bootstrap_auc(y_fail, scores, n_boot, seed)`	Labels, scores, resamples (2,000), seed	Percentile 95% interval for ROC AUC	
`perf.conv2d_flops(...)`	Input and output channels, kernel size, output height and width, groups	FLOPs of one convolution (integer)	
`perf.onnx_conv_profile(path)`	ONNX model file	Per convolution: shape, FLOPs, bytes moved, arithmetic intensity (FLOP/byte)	
`report.input_warnings(rgb)`	uint8 RGB frame	Warnings for wrong size, very dark, very bright or near-uniform frames	
`cli.resolve_threshold(arg, weights, cost_ratio)`	`auto` or a number; model file; r	τ, read from `threshold.json` for that model and r	`ValueError` if outside [0, 1] or none stored; `FileNotFoundError` if the file is missing
`cli.main(argv)`	`inspect <files or folders> [--weights] [--threshold] [--cost-ratio] [--out]`	Per-frame JSON and PNG, batch summary and HTML index; exit code 0, 1 (bad arguments) or 2 (an input was an ERROR)	Bad files become ERROR reports and the batch continues

Appendix C: Additional figures

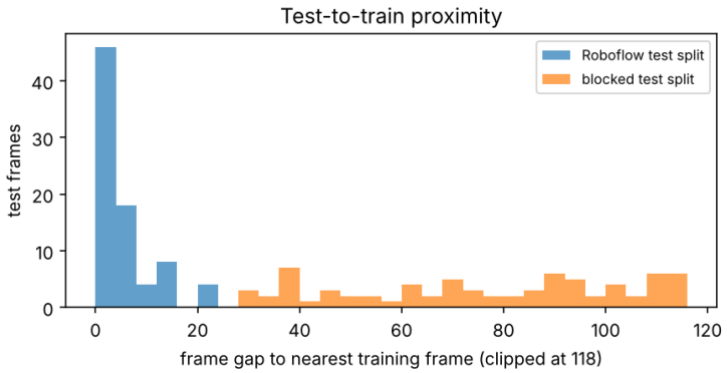


Figure C1. Gap from each test frame to its nearest training frame. Every provided test frame sits within 30 frames of a training frame; the blocked split removes this.

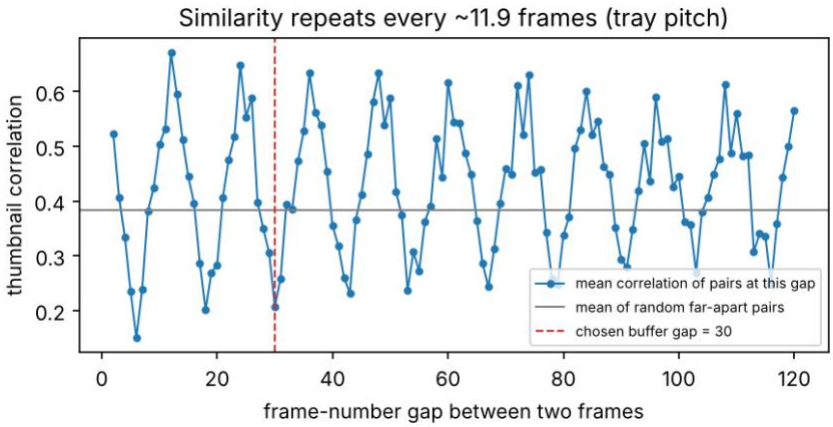


Figure C2. Correlation of frame thumbnails against frame gap. The curve repeats every 11.9 frames, the spacing between trays; the 30-frame buffer spans about 2.5 trays.

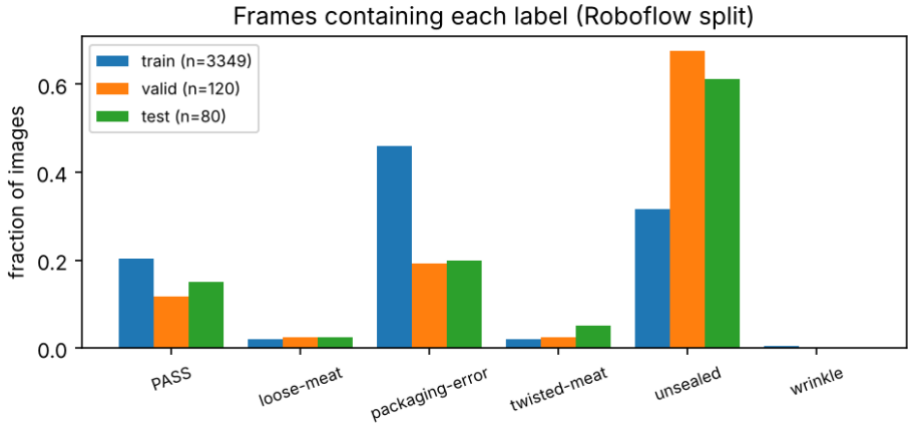


Figure C3. Share of frames containing each label in the provided splits. PASS is a minority and three classes are rare.

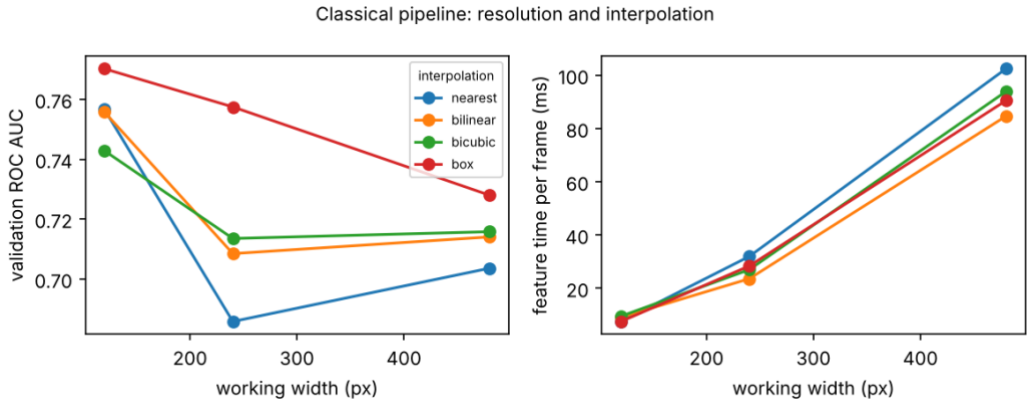


Figure C4. Classical pipeline: validation AUC (left) and feature time (right) against working width and interpolation method. Box filtering is best at every size; time grows with pixel count.

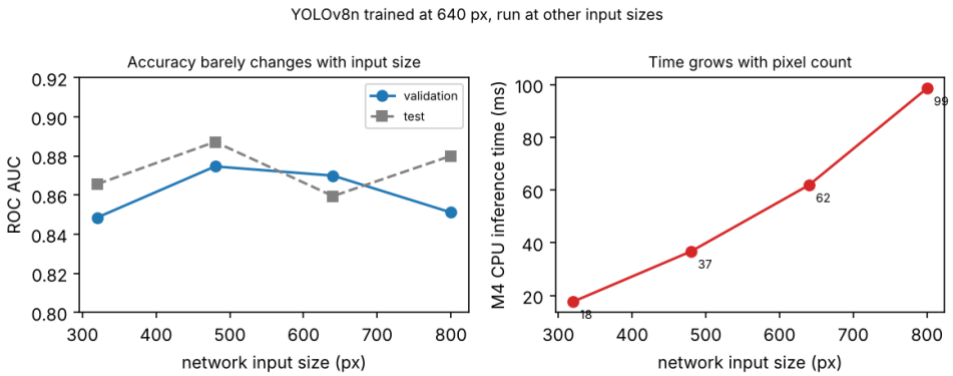


Figure C5. YOLOv8n trained at 640 px and run at other input sizes: accuracy barely changes, while time grows with pixel count.

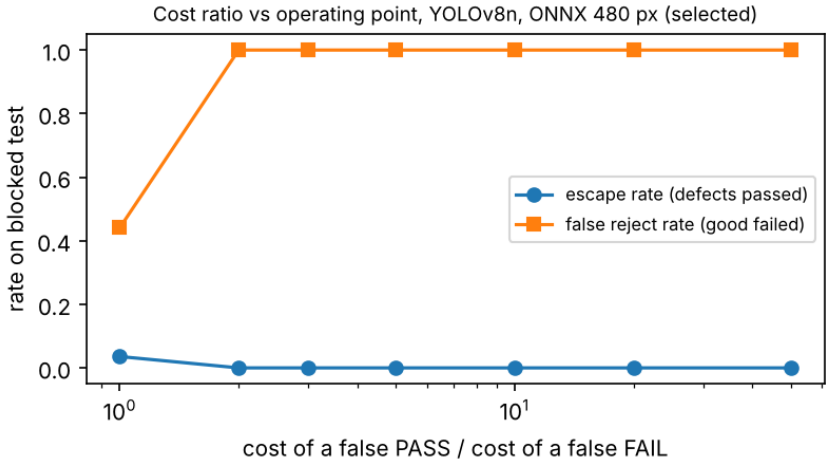


Figure C6. Selected detector: test escape rate and false reject rate at the threshold chosen on validation for each cost ratio. Only at r = 1 does the threshold pass any frames.

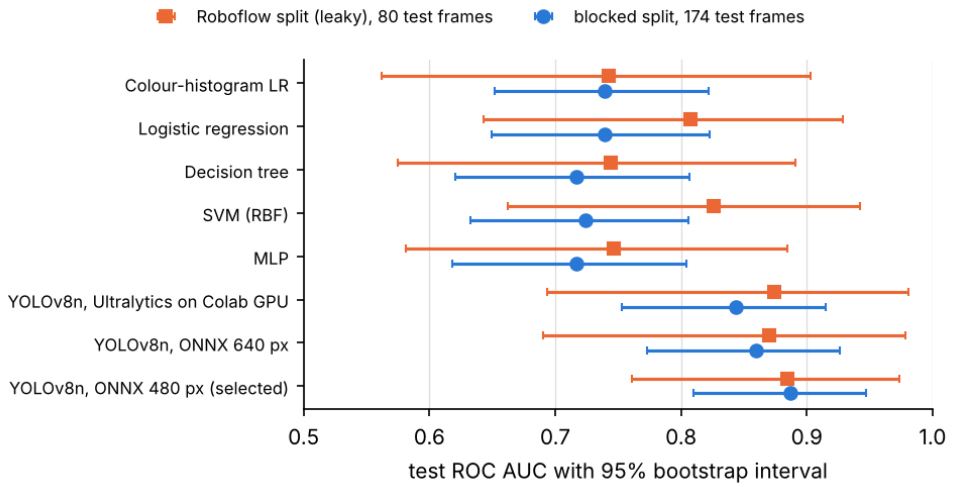


Figure C7. Test ROC AUC on the leaky provided split (80 test frames) and the blocked split (174 test frames), with 95% bootstrap intervals. Logistic regression and the SVM gain 0.07 to 0.10 on the leaky split; the detector does not change measurably.

Appendix D: Chosen settings

Model	Settings chosen by grouped cross-validation
Colour-histogram logistic regression	C = 0.01, balanced class weights
Logistic regression	C = 0.001, balanced class weights
Decision tree	maximum depth 5, at least 20 samples per leaf (18 leaves)
SVM	RBF kernel, C = 3, gamma = 1e-4 (1,606 support vectors)
MLP	one hidden layer of 64 units, L2 penalty alpha = 1.0, early stopping
YOLOv8n	640 px input, up to 60 epochs, patience 20, batch 16, seed 0, COCO-pretrained; stopped at epoch 39 with the weights from epoch 19; deployed at a 480 px input
Decision threshold	chosen on validation for each cost ratio; r = 10 by default

Appendix E: Verification tests

Test file	Tests	What they establish
`test_dataset.py`	10	Label files are parsed and malformed ones rejected with the file and line; augmented copies group to one source frame
`test_splits.py`	6	Whole blocks stay together, no test frame lies within the gap of a training frame, evaluation uses one copy per frame, the split is reproducible
`test_decision_evaluate.py`	8	The PASS/FAIL rule; the threshold search equals a brute-force search and never rises as the cost ratio rises; Wilson intervals match known values
`test_features_detector.py`	13	Feature vectors have a fixed length; letterbox, IoU and NMS give known answers; the decoder recovers a hand-encoded box; the FLOP formula matches a loop count; the optimised histogram and HSV conversion match the references
`test_onnx_pipeline.py`	4	A small synthetic ONNX network runs end to end in fp32 and fp16, and the graph FLOP profiler matches the formula
`test_report_cli.py`	5	Reports are written; corrupt, non-image and invalid inputs become ERROR reports without stopping a batch; thresholds are looked up by model and cost ratio
Doctests in `src/`	6	Worked examples in the docstrings

Appendix F: Environment

Development and classical models: Python 3.13, scikit-learn 1.9.1, scikit-image 0.26.0, NumPy 2.5.3, pandas 3.0.5, ONNX Runtime 1.29.0, Pillow 12.3.0. Timing on the target laptop: Apple MacBook with M4 chip, in a 4-core Linux virtual machine with Python 3.10. Training: Google Colab with an NVIDIA T4 GPU, Ultralytics 8.4.185, PyTorch 2.11.0 (CUDA 13.0), Python 3.13.

Appendix G: Validation and performance details

ONNX pipeline against Ultralytics (Section 5.1). Two experiments traced the remaining score differences (0.04 on average over 60 frames) to the input rather than the decoding. Resizing with OpenCV instead of Pillow barely changed them, while moving the padding from centred to top-aligned shifted scores by 0.024, because Ultralytics feeds a 640 x 480 rectangle where the exported model needs a padded square.

Roofline inputs (Section 6.2). The M4 peak uses the unit's formula: 4 performance cores x 4.4 GHz x 4 lanes (128-bit vectors of fp32) x 2 (fused multiply-add) x 4 pipes ≈ 563 GFLOP/s [16], the four pipes being

an estimate. With about 120 GB/s of memory bandwidth the machine balance is 4.7 FLOP/byte. The T4 offers 8.1 TFLOP/s in fp32 and 320 GB/s [17], a balance of about 25 FLOP/byte. Individual convolutions range from 0.5 to 174 FLOP/byte. Four of the 64, the head’s final 1 x 1 output layers with only a few output channels, fall below the M4’s balance and are memory-bound, even though the network as a whole is compute-bound. The measured 140 GFLOP/s is a quarter of the peak, probably because the virtual machine's cores may not all be performance cores, small layers use the vector units poorly, and the FLOP count omits activations and decoding.